LiDAR SLAM with Multiple Object Tracking(MOT) for Autonomous Driving

Submitted by

Zhang Zhengyang

A0318487N

Supervisor：Prof. Marcelo H Ang Jr

Session 2025/2026

10 April 2026

Contents

Acknowledgment

As I complete this thesis, I realize that it also marks the end of my student life. At the age of twenty-two, having reached the final stage of my academic journey, I feel deeply grateful to everyone who has supported me along the way. This thesis is not only a summary of my research work, but also a meaningful conclusion to many years of learning and growth.

First of all, I would like to express my sincere gratitude to all the teachers in the College of Design and Engineering during my master's study at the National University of Singapore. Over the past year, I have benefited greatly from their teaching, guidance, and academic support. Their patience and dedication have helped me deepen my understanding of both research and engineering practice.

I am especially grateful to my supervisor, Professor Marcelo H. Ang Jr., for his valuable guidance and support throughout this project. I would also like to sincerely thank my examiner, Professor Teo Chee Leong, for his time and thoughtful evaluation of this work. My heartfelt thanks also go to Han Yuhang, whose guidance was of great help during this research, and to every member of our research group for their companionship, discussions, and encouragement.

Finally, I would like to express my deepest respect and gratitude to my family and friends. Their love, trust, and support have always given me strength, especially during difficult and uncertain moments.

Life is still a long journey, and the future holds endless possibilities. As I move forward into work and life beyond school, I will carry this kindness with me and continue my journey with sincerity, gratitude, and hope.

Abstract

SLAM (Simultaneous Localization and Mapping) has become a fundamental technique for autonomous driving and robotic navigation due to its critical role in localization and environmental mapping [4]. Among LiDAR-based SLAM methods, FAST-LIO2 (Fast Direct LiDAR-inertial Odometry [3]) demonstrates excellent real-time performance by directly registering raw point clouds to the map and tightly coupling LiDAR and IMU (Inertial Measurement Unit) measurements. However, in dynamic scenes, moving objects such as vehicles and pedestrians introduce non-static geometric structures into the environment, which may degrade scan matching quality and lead to inaccurate trajectory estimation. Traditional approaches often treat dynamic objects as outliers and simply remove them, but such strategies may discard useful motion-related information and remain insufficient in complex traffic environments.

To address this issue, this paper proposes a trajectory optimization method for FAST-LIO2 in dynamic scenes via dynamic-object weight optimization and joint optimization. The proposed framework integrates object detection and MOT (Multi-Object Tracking) with the original FAST-LIO2 pipeline. In the front end, tracked object information is used to identify dynamic regions and assign adaptive weights to different point cloud observations, thereby reducing the negative influence of moving objects on pose estimation while preserving

informative scene structure. In the back end, a joint optimization strategy is further introduced to refine ego trajectory estimation within a sliding window. Specifically, odometry consistency constraints and object observation consistency constraints are jointly constructed, enabling tracked object observations to provide additional guidance for trajectory correction.

Experiments on representative dynamic driving sequences demonstrate that the proposed method can effectively improve trajectory estimation accuracy compared with the baseline FAST-LIO2. The results further show that the method is especially beneficial in scenes with stronger dynamic interference, where front-end weight optimization and back-end joint optimization complement each other to enhance robustness and stability.

Keywords: SLAM; LiDAR-inertial odometry; FAST-LIO2; dynamic scenes; trajectory optimization; weight optimization; joint optimization

Chapter 1 Introduction

Section 1.1 Research Background

SLAM has become one of the fundamental technologies in autonomous driving and robotic navigation, as it enables a system to estimate its own motion while constructing a representation of the surrounding environment [12]. In practical intelligent driving systems, accurate localization is essential for perception, planning, and control. If the estimated trajectory is unreliable, errors may propagate to subsequent modules and reduce the overall stability of the system. Therefore, localization and mapping remain key research topics in autonomous systems and continue to attract broad attention in both academia and engineering applications.

Among different sensing modalities, LiDAR-based methods have shown significant advantages in outdoor localization and mapping tasks. Compared with cameras, LiDAR sensors directly provide three-dimensional geometric information of the environment and are less sensitive to illumination changes and visual disturbances. This makes them particularly suitable for autonomous driving scenarios, where vehicles need to operate in roads, residential areas, intersections, and other complex outdoor environments. Through continuous laser scanning, LiDAR can capture the spatial distribution of surrounding structures and generate point cloud data for pose estimation and map

construction.

In many practical systems, LiDAR is further combined with inertial sensors to improve state estimation performance. The IMU provides high-frequency motion measurements, which help maintain motion continuity and compensate for the temporal limitations of LiDAR scans. By fusing LiDAR observations with inertial information, LiDAR-inertial methods can achieve better real-time performance and more stable pose estimation than LiDAR-only approaches. As a result, LiDAR-inertial localization and mapping has become an important research direction in autonomous driving and robotics.

Within this research direction, FAST-LIO2 is a representative method because of its efficiency and strong localization capability. Unlike conventional approaches that rely heavily on handcrafted feature extraction, FAST-LIO2 directly registers raw point clouds to the map and tightly couples LiDAR and inertial measurements for state estimation. This design preserves more geometric information from the original point cloud while also improving computational efficiency. In addition, its efficient map management strategy supports high-frequency processing and makes the framework suitable for real-time applications. Owing to these advantages, FAST-LIO2 has been widely regarded as an effective baseline for LiDAR-inertial odometry research.

However, real autonomous driving environments are often highly dynamic. Urban roads usually contain moving vehicles, pedestrians, cyclists, and other

traffic participants, while the spatial arrangement of the scene may change continuously due to traffic flow, temporary parking, or occlusion. These factors make real-world localization conditions much more complex than those in ideal static settings. Consequently, research on LiDAR SLAM in dynamic scenes has become increasingly important. For work built upon FAST-LIO2, improving trajectory estimation under such conditions is not only a meaningful extension of existing LiDAR-inertial research, but also of practical value for achieving more robust localization in realistic traffic environments.

Section 1.2 Problem Statement and Motivation

Although FAST-LIO2 has demonstrated strong performance in LiDAR-inertial odometry, its trajectory estimation accuracy may still degrade in dynamic scenes [5]. The core reason is that real traffic environments are not fully static. In practical autonomous driving scenarios, point clouds often contain observations from moving vehicles, pedestrians, cyclists, and other traffic participants. These dynamic elements continuously change their positions over time, which weakens the geometric consistency between consecutive observations and increases the difficulty of stable pose estimation.

For LiDAR-based localization, reliable trajectory estimation depends heavily on the assumption that most observed structures remain unchanged during motion. However, this assumption is often violated in urban

environments. When moving objects occupy a noticeable portion of the sensor's field of view, their point cloud observations may be mixed with static background structures during registration and map update [6]. As a result, the estimated motion of the ego vehicle can be affected by non-static geometric information, which may lead to local pose deviation and accumulated trajectory error. In scenes with dense traffic participants or frequent relative motion, this problem becomes more significant.

Another challenge lies in the complexity of dynamic scenes themselves. Dynamic interference is not limited to high-speed moving objects. Temporary stopping vehicles, slowly moving traffic participants, partial occlusions, and changes in local observation density may all influence the stability of point cloud registration [13]. In such cases, trajectory estimation errors may not come from a single obvious source, but from the combined effect of multiple dynamic factors. Therefore, improving localization performance in dynamic scenes requires more careful consideration of how scene changes affect geometric constraints in the odometry process.

From the perspective of practical autonomous driving, this issue is especially meaningful. Accurate ego trajectory estimation is a prerequisite for environmental perception, behavior planning, and motion control [1]. If localization becomes unstable in scenes with frequent dynamic interference, the reliability of the whole autonomous system may be reduced. Therefore, there is

strong motivation to further study how FAST-LIO2 can be improved for dynamic environments, so that it can maintain better trajectory quality under more realistic road conditions.

Based on the above considerations, this work focuses on the trajectory estimation problem of FAST-LIO2 in dynamic scenes. The main motivation is to improve localization robustness and trajectory accuracy in the presence of dynamic interference, while preserving the efficiency and practical advantages of the original FAST-LIO2 framework. This problem setting is both theoretically meaningful for dynamic-scene LiDAR-inertial research and practically relevant for real-world autonomous driving applications.

Section 1.3 Related Development of Dynamic-Scene LiDAR SLAM

LiDAR SLAM and LiDAR-inertial odometry have been extensively studied for autonomous driving and robotic navigation. Early representative methods such as LOAM [2] established an effective framework for real-time lidar odometry and mapping by separating high-frequency odometry estimation from lower-frequency mapping. Building on this research line, later methods further improved computational efficiency, robustness, and estimation accuracy. Among them, FAST-LIO2 [3] has become a widely adopted baseline due to its direct registration of raw points to the map, tight coupling of LiDAR and inertial measurements, and efficient incremental map management. These

methods have demonstrated strong performance in structured environments and provided an important foundation for subsequent dynamic-scene research.

However, traditional LiDAR SLAM methods are mainly designed under the assumption that most observed structures are static. When deployed in urban traffic environments, moving vehicles, pedestrians, and other dynamic objects may introduce unstable geometric observations into the localization process. To address this issue, one major research direction has focused on identifying and removing dynamic observations from point clouds or maps. Representative methods such as Removert [7] and ERASOR [8] aim to construct static point cloud maps in dynamic urban environments by detecting dynamic regions and filtering out moving-object traces. These studies show that dynamic-object suppression can improve the consistency of static map construction and reduce the negative impact of moving objects on subsequent localization.

With the increasing demand for robust localization in realistic traffic scenarios, more recent studies have extended this idea from map-oriented dynamic removal to dynamic-scene LiDAR-inertial odometry. For example, some works explicitly incorporate moving-object detection into the odometry pipeline to improve pose estimation in dynamic scenes. A representative example is the work by Hu et al. [9] , which combines LiDAR-inertial odometry with moving-object detection for dynamic environments. This type of research indicates that dynamic-scene localization is not only a map-cleaning

problem, but also a trajectory estimation problem in which dynamic observations directly influence odometry quality.

Another related development is the tighter integration of localization and object-level dynamic analysis. DL-SLOT [10] , for example, formulates dynamic lidar SLAM and object tracking within a unified optimization framework, showing that ego-motion estimation and dynamic-object tracking can be treated as closely related problems in road scenarios. This line of work suggests that, beyond suppressing dynamic interference, structured information associated with moving objects may also play a role in improving localization robustness.

Overall, existing research shows a clear development path in dynamic-scene LiDAR SLAM: from classical geometric odometry and mapping, to dynamic point removal and static map recovery, and then to methods that address dynamic-scene trajectory estimation more directly. Recent review work has also summarized this trend and highlighted the growing importance of dynamic-object filtering in LiDAR-based SLAM systems [11] . This development provides important background for further improving FAST-LIO2 in dynamic scenes and forms the research basis of the present work.

Section 1.4 Main Idea and Contributions of This Work

This work focuses on trajectory optimization for FAST-LIO2 in dynamic scenes. The proposed framework combines object detection, multi-object tracking, odometry estimation, and trajectory refinement into a unified pipeline for autonomous driving scenarios. Its main purpose is to improve ego trajectory estimation under dynamic interference while preserving the efficiency of the original FAST-LIO2 framework.

The workflow starts from front-end perception. For each frame, the system first performs 3D object detection on the point cloud and then uses multi-object tracking to maintain temporal consistency of the detected vehicles. At the same time, FAST-LIO2 processes LiDAR and IMU data to estimate the ego pose. In this way, the framework obtains both object-level scene information and real-time odometry information from the same driving process.

Based on these outputs, the framework further connects tracking and localization for trajectory-oriented optimization. The tracked objects provide structured dynamic-scene information for FAST-LIO2, which is used for front-end weight optimization to reduce the influence of unstable regions during odometry estimation. On this basis, a back-end joint optimization stage is further introduced to refine ego trajectory estimation and improve overall trajectory consistency in dynamic scenes.

Overall, this work builds a complete dynamic-scene trajectory optimization

pipeline for FAST-LIO2. By connecting detection, tracking, front-end weight optimization, and back-end joint optimization within one framework, the proposed study extends conventional LiDAR-inertial trajectory estimation to a more structured dynamic-scene setting and provides a practical solution for improving localization robustness in realistic traffic environments.

Chapter 2 Related Theories and Key Techniques

Section 2.1 Overview of FAST-LIO2

FAST-LIO2 [3] is a representative LiDAR-inertial odometry framework designed for real-time and accurate state estimation in complex environments. By tightly coupling LiDAR measurements with inertial information, it achieves high-frequency pose estimation while maintaining strong robustness in large-scale outdoor scenes. Owing to its efficiency and reliable localization capability, FAST-LIO2 has become an important baseline for many studies related to LiDAR-inertial localization and mapping.

As illustrated in the system framework in Figure 2-1, FAST-LIO2 mainly consists of two closely connected parts: state estimation and map management. The state estimation module fuses LiDAR and inertial measurements to estimate the motion state of the platform, while the mapping module incrementally updates the local map to support point cloud registration. Through this tightly coupled design, the framework maintains a balance between estimation accuracy and real-time performance.

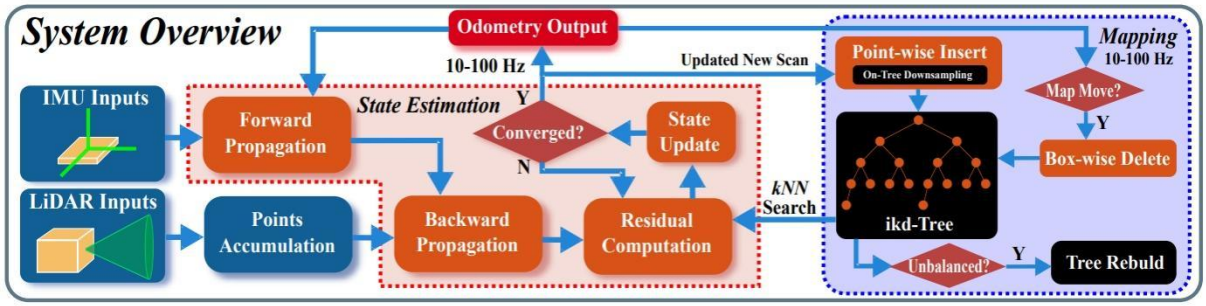

Figure 2-1 FAST-LIO2 System Architecture

The core idea of FAST-LIO2 is to directly register raw point clouds to the map instead of relying heavily on handcrafted feature extraction. Compared with conventional LiDAR odometry methods that first extract edge or planar features and then perform matching, FAST-LIO2 makes fuller use of the geometric information contained in the original point cloud. This design reduces preprocessing complexity and improves computational efficiency, which is particularly beneficial for applications requiring real-time performance. As shown in Figure 2-2, the incoming point cloud is directly aligned with the map, allowing the framework to avoid excessive dependence on manually selected geometric features.

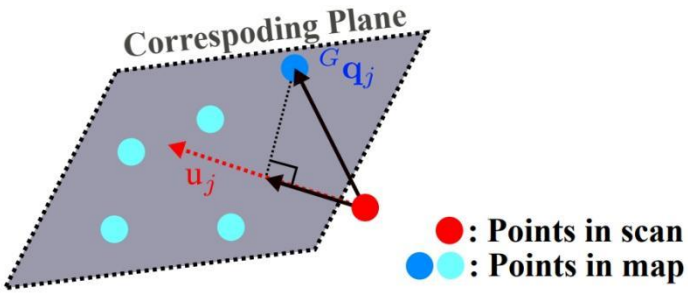

Figure 2-2 Direct Point Cloud Registration

In the FAST-LIO2 framework, the IMU provides high-frequency motion measurements for state propagation, while the LiDAR supplies geometric

observations of the surrounding environment. The inertial measurements are first used to predict the motion state of the platform, providing an initial estimate for the current scan. Then, the incoming point cloud is registered to the map, and the residuals between the observed points and the local map are used to update the system state. Through this tightly coupled process, FAST-LIO2 combines the short-term motion continuity of inertial sensing with the structural constraints of LiDAR observations, enabling accurate and stable ego-motion estimation.

Another important feature of FAST-LIO2 is its efficient map management mechanism based on the incremental k-d tree. As new LiDAR scans arrive, map points can be inserted and updated incrementally, which allows the system to maintain a local map for fast nearest-neighbor search and point-to-map registration. This strategy significantly improves runtime efficiency compared with repeatedly rebuilding the map structure from scratch. As illustrated in Figure 2-3, the incremental update strategy of the ikd-Tree enables efficient point insertion, deletion, and reorganization during mapping, thereby supporting high-frequency localization and map maintenance.

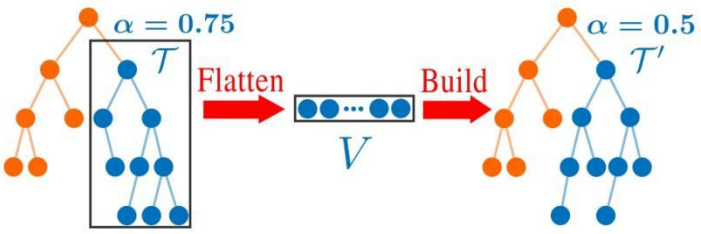

Figure 2-3 Optimization Strategy of ikd-Tree

From the perspective of system performance, FAST-LIO2 offers several important advantages. First, its direct registration strategy avoids excessive dependence on handcrafted feature extraction and makes the framework more efficient in dense outdoor environments. Second, the tight coupling between LiDAR and inertial data enhances motion estimation stability, especially during rapid movement. Third, the incremental map update strategy further improves real-time capability and makes the framework suitable for practical autonomous driving and robotic applications.

At the same time, FAST-LIO2 is still essentially developed under the assumption that most observed structures are sufficiently stable for reliable registration. In highly dynamic environments, moving vehicles and other traffic participants may introduce non-static observations into the localization process, which can influence map consistency and degrade trajectory estimation quality. For this reason, although FAST-LIO2 provides a strong and efficient baseline, further trajectory-oriented optimization is still meaningful when the framework is deployed in dynamic driving scenes.

Overall, FAST-LIO2 provides the fundamental odometry and mapping basis for the present work. Its efficient LiDAR-inertial coupling, direct point-to-map registration, and incremental map management make it well suited as the core localization framework upon which further dynamic-scene trajectory optimization can be built.

Section 2.2 Principles of 3D Object Detection

3D object detection is an important component in autonomous driving perception, since it provides structured information about surrounding traffic participants such as vehicles, pedestrians, and cyclists. Compared with raw point cloud observations, detection results offer a more compact and interpretable representation of the scene, including object category, position, size, and orientation. Such information is essential for subsequent tasks such as object tracking, scene understanding, and dynamic-scene analysis.

In LiDAR-based autonomous driving systems, 3D object detection aims to identify target objects directly from point cloud data and estimate their bounding boxes in three-dimensional space. Unlike image-based detection, LiDAR detection relies on geometric measurements rather than texture or color information, which makes it more robust to illumination changes and more suitable for outdoor environments [16] . However, point clouds are sparse, irregular, and non-uniformly distributed, which makes 3D detection more challenging than conventional 2D image detection.

To address these issues, many modern LiDAR detection methods first transform raw point clouds into a more regular representation before feature extraction. In this work, the detection module is built upon PointPillars [14] , which is a widely used real-time 3D object detection framework for point cloud data. The main idea of PointPillars is to divide the point cloud into vertical

columns, referred to as pillars, in the bird's-eye-view plane. Points within each pillar are encoded into a learned feature representation, allowing irregular point cloud data to be converted into a pseudo-image structure that can be efficiently processed by two-dimensional convolutional neural networks.

After pillar encoding, spatial features are extracted through a backbone network to generate high-level representations of the scene. Based on these features, the detection head predicts the category and 3D bounding box parameters of target objects. The output typically includes the class label, object location, box dimensions, and orientation. Through this process, PointPillars achieves a balance between computational efficiency and detection accuracy, making it suitable for autonomous driving applications that require real-time perception.

For the present work, 3D object detection serves as the first step in obtaining object-level scene information from dynamic driving environments. The detected vehicles provide the basis for subsequent multi-object tracking, enabling the system to establish temporal consistency of surrounding targets across consecutive frames. In this sense, the detection module is not only a perception component, but also an important foundation for later trajectory-oriented processing in dynamic scenes.

Overall, 3D object detection transforms unstructured point cloud observations into structured object representations and provides essential input

for downstream tracking and dynamic-scene analysis. Its role in this work is to bridge raw LiDAR perception and higher-level trajectory optimization by supplying reliable target observations from the driving environment.

Section 2.3 Principles of Multi-Object Tracking

Multi-object tracking is an important component in autonomous driving perception, since it establishes temporal consistency for detected objects across consecutive frames. Compared with single-frame detection, tracking not only preserves object identities over time but also provides motion-related information that is essential for downstream tasks such as trajectory prediction and planning. In this work, the tracking module is built upon MCTrack [15] , which is a unified 3D multi-object tracking framework designed for autonomous driving scenarios.

MCTrack follows the tracking-by-detection paradigm. Its input consists of 3D detection results that have been converted into a unified data format, and the entire tracking pipeline is carried out in the world coordinate system. The core objective is to associate current detections with existing trajectories and maintain stable target identities over time. In this way, isolated detection results from individual frames are transformed into continuous target trajectories with interpretable motion states.

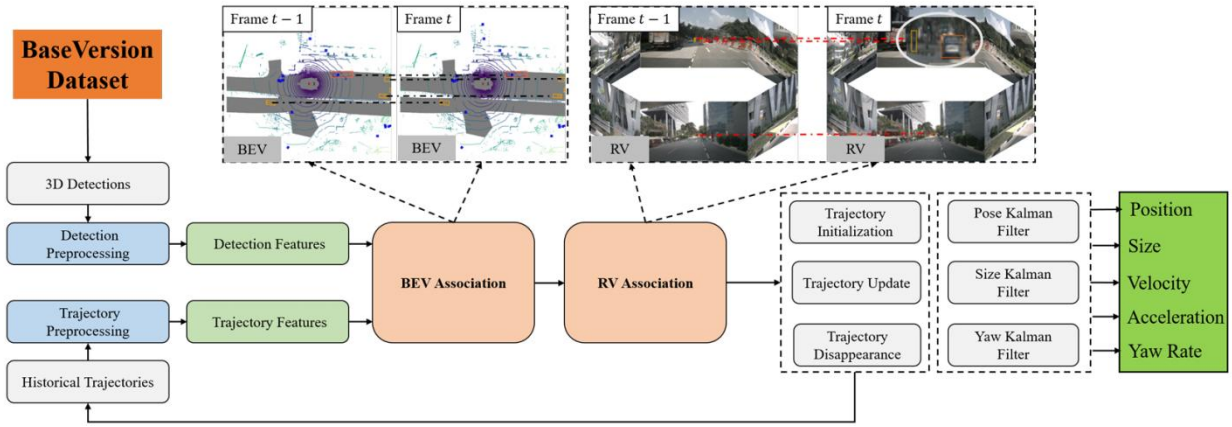

Figure 2-4 Overview of 3D MOT framework MCTrack

A key feature of MCTrack is its hierarchical association strategy. According to the framework presented in Figure 2-4, the first-stage matching is performed in the bird's-eye-view plane, where trajectory boxes and detection boxes are associated in the world coordinate system. After this primary matching step, unmatched trajectories and detections are further projected onto the image plane for secondary matching. This two-stage design improves association robustness in autonomous driving scenes, especially when targets exhibit complex motion or partial ambiguity in a single representation space.

After data association, the tracking states are updated together with the motion model. MCTrack adopts Kalman-filter-based state estimation to maintain and update the motion states of targets across frames. Through prediction and update, the tracker can preserve trajectory continuity even when detections vary over time. In addition to target identity maintenance, the framework also outputs motion-related information such as position, velocity, and acceleration, which is emphasized in the original paper as being important

for downstream autonomous driving tasks.

For the present work, multi-object tracking plays a bridging role between 3D object detection and trajectory optimization. Detection provides object observations for each frame, while tracking further organizes them into temporally consistent target-level information. As a result, the system can obtain more stable dynamic-scene cues than those provided by single-frame detections alone. These tracked objects then serve as an important basis for subsequent processing in dynamic-scene trajectory optimization.

Chapter 3 System Framework and Data Flow Design

Section 3.1 Overall System Architecture

The proposed system is built as a dynamic-scene trajectory optimization framework based on FAST-LIO2. Its objective is to improve ego trajectory estimation in autonomous driving scenarios by integrating 3D object detection, multi-object tracking, front-end weight optimization, and back-end trajectory refinement into a unified pipeline. Instead of treating perception and localization as independent processes, the framework establishes explicit interactions among these modules so that dynamic-scene information can be incorporated into trajectory estimation more effectively.

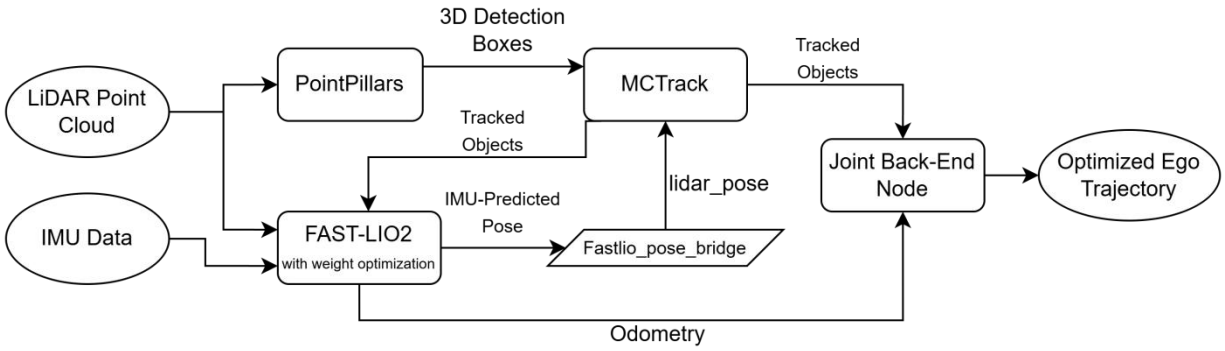

Figure 3-1 Overall architecture of the proposed dynamic-scene trajectory optimization framework

As illustrated in Figure 3-1, the system takes LiDAR point clouds and IMU data as the main inputs. The LiDAR point cloud is sent to the PointPillars module for 3D object detection, while the LiDAR and IMU measurements are

jointly processed by FAST-LIO2 for LiDAR-inertial odometry. In this way, the framework simultaneously obtains object-level observations of the environment and ego-motion estimation of the platform from the same driving sequence.

The detection results generated by PointPillars are then forwarded to MCTrack, which performs multi-object tracking to maintain the temporal consistency of surrounding vehicles across consecutive frames. At the same time, FAST-LIO2 publishes an IMU-predicted ego pose, which is converted by the fastlio_pose_bridge module and provided to MCTrack as the pose reference for transforming detections into the global frame. Based on the combination of detection boxes and ego-pose information, MCTrack produces tracked object states that can be further used in the subsequent optimization process.

A key feature of the proposed architecture is the closed-loop interaction between MCTrack and FAST-LIO2. The tracked objects produced by MCTrack are fed back to FAST-LIO2 and used for dynamic-point weight optimization in the current odometry process. Since tracking results are generated with processing delay, the framework uses the most recently available tracked objects rather than enforcing strict same-frame feedback. This design allows dynamic-scene information to be incorporated into FAST-LIO2 in a practical and temporally tolerant manner.

In addition to the front-end feedback, the framework also includes a joint back-end node for ego trajectory refinement. This node takes the odometry

output of FAST-LIO2 together with tracked object observations as input, and then performs back-end processing to generate an optimized ego trajectory. As a result, the whole system contains two trajectory-oriented stages: front-end weight optimization for reducing the influence of dynamic objects during odometry estimation, and back-end refinement for further improving trajectory consistency.

Overall, the proposed architecture forms a complete pipeline that connects perception, tracking, odometry, feedback, and trajectory optimization within one system. This structure provides the foundation for the methods described in the following sections.

Section 3.2 Bidirectional Fusion Between MCTrack and FAST-LIO2

A central characteristic of the proposed framework is the bidirectional fusion between MCTrack and FAST-LIO2. Instead of operating as two isolated modules, tracking and odometry are connected through a closed-loop interaction, so that ego-motion information can support target tracking, while tracked object information can in turn support dynamic-scene trajectory optimization.

The first direction of this interaction is from FAST-LIO2 to MCTrack. During odometry estimation, FAST-LIO2 publishes an IMU-predicted ego pose before the LiDAR update step. This pose is further converted by the

fastlio_pose_bridge module and provided to MCTrack as the ego-pose reference. In the tracking pipeline, this reference is used to transform LiDAR detections into the global frame, so that detections from consecutive frames can be represented in a consistent coordinate system. In this way, MCTrack is supported not only by frame-wise detection outputs, but also by ego-motion information from the odometry side.

The second direction of the interaction is from MCTrack back to FAST-LIO2. After data association and state update, MCTrack outputs tracked object states with temporal continuity, including object position, size, orientation, and motion-related information. These tracked objects are fed back to FAST-LIO2 and used for dynamic-region-aware processing during odometry estimation. As a result, the localization module is no longer based solely on raw geometric observations, but can also make use of structured target-level information from the tracking side.

In practice, the tracking results are produced with processing delay and therefore are not strictly synchronized with the LiDAR frame currently being processed by FAST-LIO2. For this reason, the framework uses the most recently available tracked object results rather than enforcing same-frame feedback. This design makes the closed-loop interaction feasible under practical experimental conditions while maintaining a stable data flow in the system.

Through this bidirectional fusion, FAST-LIO2 and MCTrack complement each other in the dynamic-scene pipeline. The odometry module provides ego-pose reference for tracking, while the tracking module provides temporally consistent dynamic-object information for odometry-related processing. This interaction forms the key connection between perception and localization in the proposed framework and provides the basis for the front-end weight optimization introduced in the following chapter.

Section 3.3 Joint Back-End Node for Ego Trajectory Refinement

In addition to the front-end feedback mechanism, the proposed framework further introduces a joint back-end node for ego trajectory refinement. This module is designed to improve trajectory consistency after front-end odometry estimation, especially in dynamic driving scenes where local disturbances may still remain even after dynamic-region-aware processing. Instead of replacing the original FAST-LIO2 odometry, the back-end node takes it as the primary motion estimate and performs an additional refinement process based on both odometry and tracked object information.

The input to the joint back-end node consists of two parts. The first part is the odometry output from FAST-LIO2, which provides the initial ego trajectory and inter-frame motion information. The second part is the tracked object

information produced by MCTrack, which contains temporally continuous observations of surrounding dynamic targets. By combining these two sources, the back-end node is able to incorporate both ego-motion estimation and target-level scene information into a unified refinement process.

The core role of this node is to organize the available observations into a trajectory-oriented optimization pipeline. On the one hand, the odometry results from FAST-LIO2 provide the basic motion continuity of the ego vehicle. On the other hand, the tracked objects provide additional scene constraints from the dynamic environment. Through this design, the back-end node extends the trajectory estimation process beyond pure front-end odometry and allows the system to further improve ego-motion consistency under dynamic conditions.

The output of the joint back-end node is an optimized ego trajectory for subsequent trajectory evaluation. In this way, the proposed framework contains two complementary trajectory-oriented stages: front-end dynamic-aware processing inside FAST-LIO2, and back-end ego trajectory refinement based on odometry and tracked object observations. This layered design improves the overall robustness of the system in dynamic scenes.

Section 3.4 Semi-Synchronous Offline Frame Scheduling Mechanism

To reduce frame dropping and temporal mismatch caused by the relatively long processing time of object detection and tracking, the proposed system

adopts a semi-synchronous offline frame scheduling mechanism during experiments. Instead of replaying all sensor data continuously at a fixed speed, the system controls the release of LiDAR frames in a more coordinated way, so that the perception branch can provide more complete results for each frame.

The basic idea is straightforward. After one LiDAR frame is released, the system does not immediately send out the next LiDAR frame. Instead, it first waits for the corresponding detection and tracking results of the current frame. In this way, the tracking branch is given enough time to process the current input, which helps reduce the situation where later modules receive missing or severely delayed object information.

However, this waiting process is not completely strict. The system mainly waits for the slower perception branch, especially detection and tracking, because these modules require much more processing time than odometry. By contrast, FAST-LIO2 and the joint back-end node are not forced to stop in the same frame-by-frame manner. Therefore, the scheduling mechanism is only semi-synchronous: it coordinates the release of LiDAR frames with perception results, but it does not fully block the entire pipeline.

A timeout strategy is also introduced to prevent the system from waiting indefinitely. If the required detection or tracking result for the current frame is not received within the allowed time, the system stops waiting and releases the next LiDAR frame. This design makes the pipeline more robust in practice,

because occasional slow inference or missing outputs will not cause the whole experiment to stall.

This mechanism is mainly intended for offline experiments, where the detection and tracking modules are the major sources of delay. By controlling the LiDAR frame release in this way, the framework achieves a better balance between temporal consistency and execution efficiency, which makes the full pipeline more stable for dynamic-scene trajectory optimization experiments.

Chapter 4 Front-End Weight Optimization Based on Tracked Dynamic Objects

Section 4.1 Dynamic Region Identification from Tracking Results

Section 4.1.1 Tracked Object Snapshot for Current-Frame Processing

In the proposed framework, dynamic-region identification for the current LiDAR frame is performed using the most recently available tracked object results. Due to the processing delay of object detection and multi-object tracking, these results are typically not strictly synchronized with the LiDAR frame currently being processed by FAST-LIO2, and in practice they usually correspond to the previous processing cycle. Before the current frame enters front-end processing, the available tracked objects are copied to form a fixed snapshot for that frame. This design ensures that all points in the same LiDAR frame are evaluated against a consistent set of tracked object states during dynamic-region identification.

The motivation for this design comes from the asynchronous nature of the perception pipeline. Since tracking results are updated at a different time from LiDAR odometry processing, directly using continuously changing target states would reduce the consistency of front-end processing. By contrast, the snapshot strategy guarantees that one LiDAR frame corresponds to one fixed set of tracked object observations, even though these observations may originate from

the immediately preceding cycle.

Each tracked object in the snapshot contains the basic geometric and temporal information required for subsequent dynamic-region identification, including the object center, box dimensions, orientation, and timestamp. These tracked objects provide structured target-level information for the current frame and serve as the basis for determining whether a LiDAR point belongs to a dynamic region.

Section 4.1.2 Expanded Dynamic Bounding Box Representation

After the tracked object snapshot is obtained for the current LiDAR frame, each tracked object is represented as an oriented three-dimensional bounding box for dynamic-region identification. A tracked object j is characterized by its box center

$$\mathbf{c}_j = [c_x^{(j)}, c_y^{(j)}, c_z^{(j)}]^{\mathrm{T}} \qquad (4-1)$$

its box dimensions

$$\mathbf{d}_j = [d_x^{(j)}, d_y^{(j)}, d_z^{(j)}]^{\mathrm{T}} \qquad (4-2)$$

and its yaw angle θ_j, which describes the object orientation in the horizontal plane. Based on these quantities, the tracked object can be modeled as an oriented 3D box in the global frame.

In the proposed framework, the original tracked bounding box is not used directly for dynamic-region identification. Instead, a fixed spatial margin is

added to the box dimensions to obtain an expanded dynamic bounding box. This design is introduced to improve robustness under practical conditions, since the tracked objects used for the current LiDAR frame usually come from the most recently available tracking results rather than strictly synchronized same-frame outputs. As a result, small temporal delay, localization fluctuation, or target motion between two consecutive processing cycles may lead to spatial mismatch between the tracked box and the actual point distribution in the current frame.

To tolerate such mismatch, the half size of the expanded bounding box is defined as

$$\mathbf{h}_j = \begin{bmatrix} \frac{d_x^{(j)}}{2} + \Delta_{xy} \\ \frac{d_y^{(j)}}{2} + \Delta_{xy} \\ \frac{d_z^{(j)}}{2} + \Delta_z \end{bmatrix} \qquad (4-3)$$

Where Δ_{xy} denotes the additional margin in the horizontal plane and Δ_z denotes the additional margin in the vertical direction. In the current implementation, the horizontal margin is set to 0.6 m and the vertical margin is set to 0.5 m.

With this representation, each tracked object defines an expanded oriented spatial region around the estimated target position. Compared with using the original bounding box directly, this expanded representation provides a more

robust basis for dynamic-region identification in the front-end processing of FAST-LIO2. It allows the system to maintain tolerance to moderate delay and spatial deviation while still preserving the target-level geometric structure required for point-wise region testing. Based on the expanded dynamic bounding box, the next subsection further determines whether a LiDAR point belongs to the dynamic region of a tracked object.

Section 4.1.3 Point-in-Box Test for Dynamic Region Identification

To determine whether a LiDAR point in the current frame belongs to the dynamic region of a tracked object, a point-in-box test is performed based on the expanded dynamic bounding box. Since the tracked box is oriented by its yaw angle, this test is carried out in the local coordinate system of the object rather than directly in the global frame.

Let

$$\mathbf{p}_i = [x_i, y_i, z_i]^{\mathrm{T}} \qquad (4-4)$$

denote the i-th LiDAR point in the current frame, and let c_j be the center of the j-th tracked object. The first step is to compute the relative position of the point with respect to the box center:

$$\Delta x_i^{(j)} = x_i - c_x^{(j)}, \quad \Delta y_i^{(j)} = y_i - c_y^{(j)}, \quad \Delta z_i^{(j)} = z_i - c_z^{(j)} \qquad (4-5)$$

Next, the point is rotated into the local coordinate system of the tracked object according to its yaw angle θ_j. The transformed horizontal coordinates

are given by

$$x_{i,\mathrm{loc}}^{(j)} = \cos\theta_j \, \Delta x_i^{(j)} + \sin\theta_j \, \Delta y_i^{(j)}, \qquad (4-6\mathrm{a})$$

$$y_{i,\mathrm{loc}}^{(j)} = -\sin\theta_j \, \Delta x_i^{(j)} + \cos\theta_j \, \Delta y_i^{(j)} \qquad (4-6\mathrm{b})$$

Based on the expanded half size

$$\mathbf{h}_j = [h_x^{(j)}, h_y^{(j)}, h_z^{(j)}]^{\mathrm{T}} \qquad (4-7)$$

the point is regarded as lying inside the dynamic region of object j if the following conditions are simultaneously satisfied:

$$\left|x_{i,\mathrm{loc}}^{(j)}\right| \le h_x^{(j)}, \quad \left|y_{i,\mathrm{loc}}^{(j)}\right| \le h_y^{(j)}, \quad \left|\Delta z_i^{(j)}\right| \le h_z^{(j)} \qquad (4-8)$$

In other words, after transforming the point into the local box frame, the system checks whether the point falls within the expanded box range along all three axes. If the above conditions hold, the point is identified as belonging to a dynamic region and will be processed differently from static background points in the subsequent weighting stage.

In addition to the geometric test, a temporal validity condition is also applied to tracked objects. Since the snapshot used by the current LiDAR frame may originate from the previous processing cycle, a tracked object is considered valid only when its timestamp is sufficiently close to the current scan time. Let t_{scan} denote the timestamp of the current LiDAR frame and t_j denote the timestamp of the j-th tracked object. Then the object is used only if

$$t_{\mathrm{scan}} - t_j \le \tau \qquad (4-9)$$

Where τ is the maximum allowable delay. In the current implementation,

τ is set to 1.0s. This condition prevents outdated tracking results from participating in dynamic-region identification.

Through the above point-in-box test, the current LiDAR frame is divided into points associated with tracked dynamic objects and points belonging to the remaining scene. This provides the direct basis for the adaptive weight assignment strategy described in the next section.

Section 4.2 Weight Assignment Strategy for Dynamic Points

Section 4.2.1 Object-Level Weight from Tracking Attributes

Once dynamic regions have been identified from tracked objects, the framework proceeds to assign a weight to each tracked object before point-level weighting is performed. In the proposed method, the weight of a dynamic object is determined according to its motion and tracking attributes rather than being assigned as a fixed constant. This design allows the weighting process to better reflect the reliability and dynamic intensity of different targets.

For the j-th tracked object, its motion magnitude is first represented by the speed norm and the acceleration norm:

$$v_j = \|\mathbf{v}_j\|_2, \quad a_j = \|\mathbf{a}_j\|_2 \qquad (4-10)$$

Where $\mathbf{v}_j$ and $\mathbf{a}_j$ denote the velocity vector and acceleration vector of the tracked object, respectively.

To make different attributes comparable in the weighting process, the speed,

acceleration, and detection score are normalized into the interval [0,1]:

$$\phi_v^{(j)} = \text{clip}\left(\frac{v_j}{v_{\text{ref}}}, 0,1\right), \quad \phi_a^{(j)} = \text{clip}\left(\frac{a_j}{a_{\text{ref}}}, 0,1\right), \quad \phi_s^{(j)} = \text{clip}(s_j, 0,1) (4-11)$$

Where v_{ref} and a_{ref} are the reference speed and reference acceleration, respectively, and s_j is the detection score of the tracked object. The operator clip(·) limits the value to the range [0,1].

Based on these normalized terms, an object-level penalty is defined as

$$P_j = \lambda_v \phi_v^{(j)} + \lambda_a \phi_a^{(j)} + \lambda_s \phi_s^{(j)} \quad (4-12)$$

λ_v, λ_a and λ_s are the penalty coefficients for speed, acceleration, and score, respectively. In this formulation, a larger object speed or acceleration leads to a larger penalty, while a higher score indicates that the tracked target is more reliable and therefore its dynamic influence should be more strongly considered in the weighting process.

The final object-level weight is then defined as

$$w_j = \begin{cases} 1, & \ell_j < \ell_{\min} \\ \text{clip}(1 - P_j, w_{\min}, 1), & \ell_j \geq \ell_{\min} \end{cases} \quad (4-13)$$

Where ℓ_j denotes the track length of the j-th object, $\ell_{\min}$ is the minimum valid track length, and $w_{\min}$ is the minimum allowable dynamic weight. This formulation means that newly appeared or insufficiently stable tracked objects are not directly down-weighted. Only when the tracked object has accumulated enough temporal evidence does the system apply adaptive suppression according to its motion and confidence attributes.

After empirical tuning, the coefficients and reference parameters used in the current implementation are listed in Table 4-1.

Table 4-1 Weighting parameters used in object-level dynamic suppression

Parameter	Description	Value
v_{ref}	Reference speed	10.0
a_{ref}	Reference acceleration	4.0
λ_v	Speed penalty coefficient	0.5
λ_a	Acceleration penalty coefficient	0.3
λ_s	Score penalty coefficient	0.2
$\ell_{\min}$	Minimum track length	3
$w_{\min}$	Minimum dynamic weight	0.2

Through this strategy, each tracked object is assigned a dynamic-aware weight before point-level processing. The resulting object-level weight is then propagated to the points falling inside its dynamic region, as described in the next subsection.

Section 4.2.2 Point-Level Weight Assignment

After the object-level weight w_j is determined for each tracked object, the next step is to assign a weight to each LiDAR point in the current frame. In the proposed framework, point-level weighting is performed according to the spatial relationship between the point and the expanded dynamic regions identified in Section 4.1. If a point does not belong to any dynamic region, it is

treated as a static-background point and keeps the default weight. Otherwise, its weight is inherited from the associated tracked object. When a point lies inside multiple dynamic bounding boxes, the smallest object-level weight is used.

Accordingly, the point-level weight is defined as

$$w_i = \begin{cases} 1, & p_i \notin \bigcup_j \mathcal{B}_j \\ \min\limits_{j:p_i \in \mathcal{B}_j} w_j, & p_i \in \bigcup_j \mathcal{B}_j \end{cases} \quad (4-14)$$

Where $\mathcal{B}_j$ denotes the expanded dynamic bounding box of the j-th tracked object.

This strategy does not directly remove dynamic points from the current LiDAR frame. Instead, dynamic points are preserved but assigned smaller weights according to the properties of the associated tracked objects. In this way, the framework reduces the influence of unstable observations while maintaining the continuity of front-end point cloud processing. These point-level weights are then incorporated into the FAST-LIO2 front end, as described in the next section.

Section 4.3 Integration of Dynamic Weighting into FAST-LIO2

The dynamic weighting strategy is integrated into the FAST-LIO2 front end rather than implemented as an external filtering module. In the proposed framework, dynamic-region identification and weight assignment are first

completed during point preprocessing, and the resulting point-level weights are then directly carried into the subsequent odometry estimation process. In this way, dynamic-object information is incorporated into the front-end pipeline without changing the overall structure of FAST-LIO2.

At the preprocessing stage, each LiDAR point in the current frame is evaluated using the tracked object snapshot described in Section 4.1. According to the point-level weighting rule in Section 4.2, a dynamic weight is assigned to the point and written into its intensity field. In the current implementation, the dynamic weight is combined with the original point intensity by taking their minimum value, so that the influence of points in dynamic regions can be effectively suppressed before front-end optimization starts. Therefore, when the current frame enters the FAST-LIO2 front end, the point cloud is no longer treated as an unweighted raw observation set, but as a dynamically weighted point set whose effective weights are stored in the point attributes.

During odometry estimation, these point-level weights are further integrated into the measurement model of FAST-LIO2. Let I_i denote the weight stored in the intensity field of the i-th point during preprocessing. The effective front-end weight is written as

$$w_i = \max(w_{\min}, \min(1, I_i)) \qquad (4-15)$$

Where $w_{\min}$ is the minimum allowable dynamic weight. Based on this weight, the measurement Jacobian and residual of the corresponding point are

scaled as

$$\mathbf{H}_i^{'} = w_i \mathbf{H}_i \qquad (4-16)$$

$$r_i^{'} = w_i r_i \qquad (4-17)$$

$\mathbf{H}_i$ and r_i denote the original measurement Jacobian and residual of the i-th point, and $\mathbf{H}_i^{'}$ and $r_i^{'}$ denote their weighted forms. As a result, points associated with dynamic objects contribute less to the state update than static-background points. This allows the front end to suppress unstable observations while preserving the continuity of the original optimization process.

An important characteristic of this design is that dynamic points are not directly removed from the scan. Instead, they remain in the point cloud but with reduced influence in registration. Compared with hard removal, this strategy is more suitable for the current FAST-LIO2 framework because it preserves the basic geometric structure of the scan while still reducing the negative impact of moving objects. In this sense, the proposed method introduces dynamic awareness into the front-end estimation process in a soft and continuous manner.

In addition to odometry estimation, the weighting strategy also influences map quality. Points that have been heavily down-weighted are further suppressed during map saving, which reduces the possibility that strong dynamic interference is written into the final map. This helps alleviate dynamic

ghosting and makes the resulting map cleaner in dynamic scenes. Therefore, the proposed front-end weighting strategy improves not only the robustness of odometry estimation, but also the quality of map construction under dynamic conditions.

Chapter 5 Back-End Joint Optimization for Ego Trajectory Refinement

Section 5.1 Odometry Consistency Constraint

The back-end optimization is performed in a sliding-window manner, where the optimization target is the ego trajectory within the current window. Let the ego state at time step k be defined as

$$\mathbf{x}_k = \begin{bmatrix} \mathbf{t}_k \\ \psi_k \end{bmatrix}, \quad \mathbf{t}_k = [x_k, y_k, z_k]^{\mathrm{T}} \tag{5-1}$$

Where $\mathbf{t}_k$ denotes the translation of the ego vehicle and ψ_k denotes its yaw angle. Accordingly, the optimization variable in the window is written as

$$\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, ..., \mathbf{x}_N\} \tag{5-2}$$

The first type of constraint in the back-end optimization is the odometry consistency constraint. Its purpose is to preserve the motion continuity provided by FAST-LIO2 and to prevent the refined trajectory from deviating excessively from the original front-end estimation. Since the front-end odometry already provides a reliable initial trajectory, the back-end optimization is not designed to replace it, but to refine it while maintaining inter-frame consistency.

For two adjacent ego states $\mathbf{x}_k$ and $\mathbf{x}_{k+1}$, the corresponding FAST-LIO2 output provides a relative motion measurement between the two frames. Let the measured relative translation and yaw increment be denoted by

$$\hat{\mathbf{u}}_k = \begin{bmatrix} \hat{\mathbf{d}}_k \\ \widehat{\Delta\psi}_k \end{bmatrix}, \quad \hat{\mathbf{d}}_k = \begin{bmatrix} \hat{d}_{x,k} \\ \hat{d}_{y,k} \\ \hat{d}_{z,k} \end{bmatrix} \tag{5-3}$$

Where $\hat{\mathbf{d}}_k$ is the relative translation from frame k to frame k+1, and $\widehat{\Delta\psi}_k$ is the corresponding yaw increment.

Based on the optimized states, the estimated relative motion between two adjacent frames can be written as

$$\mathbf{u}_k(\mathbf{X}) = \begin{bmatrix} \mathbf{t}_{k+1} - \mathbf{t}_k \\ \psi_{k+1} - \psi_k \end{bmatrix} \tag{5-4}$$

Accordingly, the odometry consistency residual is defined as

$$\mathbf{r}_{\text{odom}}^{(k)} = \begin{bmatrix} \mathbf{t}_{k+1} - \mathbf{t}_k - \hat{\mathbf{d}}_k \\ \psi_{k+1} - \psi_k - \widehat{\Delta\psi}_k \end{bmatrix} \tag{5-5}$$

This residual measures the deviation between the optimized inter-frame motion and the odometry measurement provided by FAST-LIO2. When the residual is small, the refined trajectory remains close to the original front-end odometry; when the residual becomes large, the optimization is penalized accordingly. Therefore, this term serves as the basic prior of the back-end formulation and maintains the smoothness and continuity of the ego trajectory within the sliding window.

For a window containing N ego states, the odometry consistency term can be accumulated over all adjacent frame pairs:

$$E_{\text{odom}}(\mathbf{X}) = \sum_{k=1}^{N-1} \left\| \mathbf{r}_{\text{odom}}^{(k)} \right\|^2 \tag{5-6}$$

Through this formulation, the back-end optimization preserves the fundamental motion structure estimated by FAST-LIO2 while leaving room for additional object-based observations to further refine the trajectory. The next section introduces the object observation consistency constraint, which provides complementary dynamic-scene information for ego trajectory correction.

Section 5.2 Object Observation Consistency Constraint

In addition to odometry consistency, the back-end optimization also incorporates object observation consistency as an additional source of trajectory correction. The basic idea is that tracked objects provide temporally continuous observations of the dynamic scene, and these observations can be used to constrain the ego trajectory from another perspective. If the ego poses in the sliding window are accurate, the object observations associated with these poses should remain geometrically consistent after coordinate transformation.

For each ego state $\mathbf{x}_k$, the system searches for the temporally matched tracked-object message and extracts valid tracked objects from it. Let the j-th tracked object associated with state k be denoted by its observed position and heading in the global frame:

$$\widehat{\mathbf{o}}_{k,j} = \begin{bmatrix} \widehat{\mathbf{p}}_{k,j} \\ \widehat{\psi}_{k,j} \end{bmatrix}, \quad \widehat{\mathbf{p}}_{k,j} = \begin{bmatrix} \hat{x}_{k,j} \\ \hat{y}_{k,j} \end{bmatrix} \tag{5-7}$$

Where $\widehat{\mathbf{p}}_{k,j}$ is the tracked object position on the horizontal plane and $\widehat{\psi}_{k,j}$ is the corresponding object heading.

At the same time, for each tracked object, the system also retains its relative observation with respect to the ego vehicle. Let this local observation be denoted by

$$\mathbf{z}_{k,j}^{\mathrm{loc}} = \begin{bmatrix} \mathbf{p}_{k,j}^{\mathrm{loc}} \\ \psi_{k,j}^{\mathrm{loc}} \end{bmatrix}, \quad \mathbf{p}_{k,j}^{\mathrm{loc}} = \begin{bmatrix} x_{k,j}^{\mathrm{loc}} \\ y_{k,j}^{\mathrm{loc}} \end{bmatrix} \tag{5-8}$$

$\mathbf{p}_{k,j}^{\mathrm{loc}}$ is the object position in the local frame of the ego vehicle and $\psi_{k,j}^{\mathrm{loc}}$ is the relative heading observation.

Given the ego pose $\mathbf{x}_k$, the corresponding object position predicted in the global frame can be written as

$$\mathbf{p}_{k,j}^{\mathrm{pred}} = \mathbf{R}(\psi_k)\mathbf{p}_{k,j}^{\mathrm{loc}} + \mathbf{t}_k^{xy} \tag{5-9}$$

Where

$$\mathbf{t}_k^{xy} = \begin{bmatrix} x_k \\ y_k \end{bmatrix}, \quad \mathbf{R}(\psi_k) = \begin{bmatrix} \cos\psi_k & -\sin\psi_k \\ \sin\psi_k & \cos\psi_k \end{bmatrix} \tag{5-10}$$

Similarly, the predicted global heading of the tracked object is given by

$$\psi_{k,j}^{\mathrm{pred}} = \psi_k + \psi_{k,j}^{\mathrm{loc}} \tag{5-11}$$

Based on these predicted quantities, the object observation consistency

residual is defined as

$$\mathbf{r}_{\text{obj}}^{(k,j)} = \begin{bmatrix} \mathbf{p}_{k,j}^{\text{pred}} - \widehat{\mathbf{p}}_{k,j} \\ \text{wrap}\left(\psi_{k,j}^{\text{pred}} - \widehat{\psi}_{k,j}\right) \end{bmatrix} \quad (5-12)$$

Where $\text{wrap}(\cdot)$ denotes angle normalization to a principal interval such as $(-\pi, \pi]$. The first two components of the residual measure the position inconsistency of the tracked object in the global frame, while the last component measures the heading inconsistency.

The object observation term over the entire sliding window can then be accumulated as

$$E_{\text{obj}}(\mathbf{X}) = \sum_{k} \sum_{j \in \mathcal{O}_k} \left\| \mathbf{r}_{\text{obj}}^{(k,j)} \right\|^2 \quad (5-13)$$

$\mathcal{O}_k$ denotes the set of valid tracked objects associated with state k.

Compared with the odometry consistency term, this object observation consistency term provides complementary information from the dynamic scene. It does not replace the front-end trajectory prior, but introduces additional geometric constraints through tracked-object observations. In this way, the back-end optimization can further correct ego trajectory deviation when odometry alone is insufficient to fully suppress the influence of dynamic interference.

Section 5.3 Joint Objective Function and Optimization Parameters

Based on the odometry consistency term and the object observation consistency term defined above, the back-end refinement is formulated as a joint optimization problem over the ego trajectory in the sliding window. The objective is to preserve the continuity of the original FAST-LIO2 odometry while introducing additional corrections from tracked object observations. In this way, the refined trajectory is constrained not only by inter-frame ego motion but also by the geometric consistency of dynamic-scene observations.

In the current implementation, the odometry term serves as the basic prior of the optimization, while the object observation term is additionally controlled by a global weighting factor. Accordingly, the overall objective can be written as

$$E(\mathbf{X}) = E_{\text{odom}}(\mathbf{X}) + \lambda_{\text{obj}} E_{\text{obj}}(\mathbf{X}) \qquad (5-14)$$

Where λ_{obj} controls the influence of the object observation term relative to the odometry term.

In the practical implementation, each object residual is further weighted according to the reliability of the corresponding tracked object. Let $s_{k,j}$ be the score of object j at state k, and let $\ell_{k,j}$ be its track length. These two quantities are normalized as

$$\bar{s}_{k,j} = \mathrm{clip}(s_{k,j}, 0,1), \quad \bar{\ell}_{k,j} = \mathrm{clip}\left(\frac{\ell_{k,j} - \ell_{\min} \quad + 1}{\ell_{\mathrm{ref}}}, 0,1\right) \qquad (5-15)$$

Where $\ell_{\min}$ is the minimum valid track length and ℓ_{ref} is the normalization range for track length. In the current code, ℓ_{ref} is implemented as a fixed constant of 6.0 after threshold shifting.

The weight assigned to the corresponding object residual is then written as

$$w_{k,j}^{\mathrm{obj}} = \max\left(w_{\mathrm{obj,min}}, \sqrt{\lambda_{\mathrm{obj}}}\left(0.5\bar{s}_{k,j} + 0.5\bar{\ell}_{k,j}\right)\right) \qquad (5-16)$$

Where $w_{\mathrm{obj,min}}$ is the minimum object residual weight. This formulation means that high-confidence and long-lived tracks exert stronger influence in the object consistency term, while short or less reliable tracks contribute less to the optimization.

Accordingly, the joint optimization problem can be expressed as

$$\mathbf{X}^* = arg\min_{\mathbf{X}}\left[E_{\mathrm{odom}}(\mathbf{X}) + \sum_{k}\sum_{j\in\mathcal{O}_k}\rho\left(\left\|w_{k,j}^{\mathrm{obj}}\mathbf{r}_{\mathrm{obj}}^{(k,j)}\right\|^2\right)\right] \qquad (5-17)$$

Where $\rho(\cdot)$ is the robust loss function used in the optimization. In the current implementation, Huber loss is adopted by default.

The optimization-related parameters were tuned empirically during development, and the final settings used in the present implementation are listed in Table 5-1.

Table 5-1 Optimization-related parameters used in the joint back-end module

Parameter	Description	Value
N	Sliding-window length	8
λ_{obj}	Global weight for object observation term	0.12
$w_{\text{obj,min}}$	Minimum object residual weight	0.03
ℓ_{ref}	Track-length normalization range	6.0
Robust loss	Loss function for nonlinear optimization	Huber
f_{scale}	Robust loss scale	0.5
σ_{odom}	Fallback odometry noise$[x, y, \psi]$	[0.20, 0.20, 0.10]

This formulation combines the continuity of front-end odometry with the corrective information provided by tracked objects. As a result, the optimized trajectory remains close to the original ego-motion estimate while becoming more consistent with dynamic-scene observations in the sliding window.

Section 5.4 Valid Target Gating and Optimization Procedure

In practical dynamic scenes, tracked object observations may contain noise, unstable tracks, or temporal mismatches. Directly incorporating all tracked objects into the optimization can degrade performance and introduce additional errors. Therefore, a target gating mechanism is applied before constructing the object observation constraints, so that only reliable and temporally consistent objects are used in the back-end refinement.

The gating process first evaluates each tracked object based on its

confidence score. Only objects whose scores exceed a predefined threshold are considered for further processing. This step removes low-confidence detections that are likely to be false positives or poorly localized.

Track stability is then assessed using the track length. Objects with very short trajectories are excluded, since they usually correspond to newly initialized or unstable tracks. By enforcing a minimum track length requirement, the system favors objects with sufficient temporal consistency.

In addition, a motion-consistency constraint is introduced to filter out abnormal targets. Objects with excessively large velocity changes are rejected. Such cases often indicate tracking failure, identity switches, or incorrect motion estimation. By removing these outliers, the remaining object observations become more reliable for trajectory refinement.

Temporal alignment between ego states and tracked objects is also considered. For each state in the sliding window, the system searches for the closest tracked-object message within a limited time interval. If no suitable match is found within the allowed time difference, the corresponding object observations are discarded. This avoids introducing spatial inconsistency caused by time misalignment.

After applying the above gating steps, a filtered set of valid objects is obtained for each state. Let $\mathcal{O}_k$ represent the set of valid objects associated with state k. Only objects in $\mathcal{O}_k$ are used to construct the residuals in the

object observation consistency term.

The main gating parameters used in the implementation are summarized in Table 5-2.

Table 5-2 Target gating parameters used in the back-end module

Parameter	Description	Value
$s_{\min}$	Minimum object confidence score	0.6
$\ell_{\min}$	Minimum track length	5
$\Delta t_{\max}$	Maximum allowed time difference	0.1s
$\Delta v_{\max}$	Maximum allowed velocity change	2.0m/s

Based on the gated object set, the back-end optimization proceeds as follows. First, a sliding window is constructed from the buffered odometry states. For each state in the window, temporally aligned tracked objects are retrieved and filtered according to the gating criteria described above. Then, odometry consistency residuals and object observation residuals are constructed for all valid states and objects.

After the residuals are assembled, nonlinear least-squares optimization is performed to update the ego states in the window. The optimization is carried out using a robust loss function to suppress the influence of remaining outliers. Once the optimization converges, the refined ego trajectory is obtained.

The optimized trajectory is then published for evaluation and analysis. It should be noted that this refined trajectory is not fed back into the front-end mapping process, but is used as an improved estimation for performance

assessment in dynamic environments.

Through this gating and optimization procedure, unreliable object observations are effectively filtered out, and only stable dynamic-scene information is used to assist ego trajectory refinement.

Chapter 6 Experiment and Analysis

Section 6.1 Introduction to Experimental Data

Our method was evaluated using the KITTI dataset. The KITTI dataset (Karlsruhe Institute of Technology and Toyota Technological Institute) [17] represents one of the most benchmark multimodal sensor datasets in autonomous driving research, jointly released by the Karlsruhe Institute of Technology (Germany) and the Toyota Technological Institute at Chicago. It has been widely adopted in autonomous driving and computer vision studies. The dataset encompasses extensive driving scenarios captured in urban environments, including city streets, residential areas, and highways.

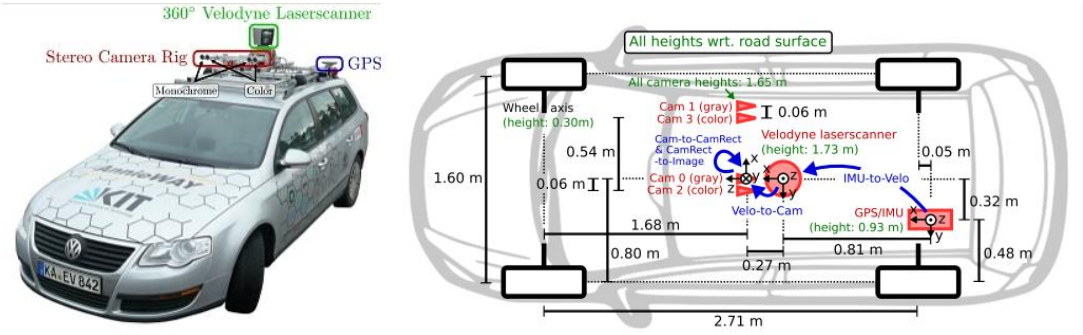

Figure 6-1 The KITTI data collection vehicle

The KITTI dataset provides multiple sensor modalities, including LiDAR point clouds, stereo images, GPS positioning information, and IMU measurements. In the present work, the proposed framework mainly uses LiDAR and IMU data for trajectory estimation and dynamic-scene optimization. The LiDAR point clouds provide the three-dimensional geometric observations

of the surrounding environment, which are used for front-end registration in FAST-LIO2 as well as object detection and tracking in the dynamic-scene pipeline. The IMU data supplies high-frequency motion information for state prediction and motion compensation. In addition, the ground-truth trajectory provided by the dataset is used for quantitative evaluation of the estimated results. Through these data sources, the KITTI dataset offers an effective benchmark for testing the proposed method under realistic autonomous driving conditions.

Section 6.2 Experimental Evaluation on KITTI Dataset

Section 6.2.1 Error Analysis Before and After Optimization

To evaluate the effectiveness of the proposed method, comparative experiments were conducted on KITTI tracking sequences. Here, the baseline denotes the trajectory estimated by the system without dynamic weighting and joint back-end optimization, while the optimized result denotes the trajectory estimated with the proposed optimization method enabled.

Table 6-1 Quantitative comparison between baseline and optimized results

Sequence		ATE RMSE (m)	RPE RMSE (m)	Translation Error (%)	Rotation Error (deg/m)
0003	Baseline	0.5294	0.2049	1.4794	1.1509
	Optimized	0.4513	0.1809	1.3516	1.1569
0006	Baseline	0.5238	0.1324	4.1614	12.8416
	Optimized	0.4710	0.0869	4.0605	11.6291
0009	Baseline	1.6831	0.1787	2.5630	3.2425
	Optimized	1.6675	0.1651	2.5281	3.2294
0010	Baseline	3.7651	0.2774	1.3884	1.1274
	Optimized	0.8645	0.2278	0.9774	1.1353
0013	Baseline	0.6047	0.1495	2.4374	2.3026
	Optimized	0.5816	0.1392	2.3741	2.2419
0020	Baseline	11.7323	0.2250	3.2245	1.1894
	Optimized	6.2936	0.2246	2.2896	1.1517

As shown in Table 6-1, the optimized results achieve lower ATE RMSE than the baseline on all tested sequences listed in this section. Averaged over these sequences, the ATE RMSE is reduced by 25.50%, while the translation error is reduced by 12.27%. The RPE RMSE and rotation error are also reduced by 13.11% and 2.41%, respectively. These results indicate that the proposed method improves trajectory estimation accuracy in both absolute and relative evaluation.

Among the tested sequences, the improvement is most obvious on Sequences 0010 and 0020. For Sequence 0010, the ATE RMSE decreases from 3.7651 m to 0.8645 m. For Sequence 0020, it is reduced from 11.7323 m to

6.2936 m. These results show that the proposed method can effectively reduce trajectory deviation in some dynamic driving scenes.

For Sequences 0003, 0006, 0009, and 0013, the improvements are relatively smaller, but the optimized results still remain better than the baseline in terms of ATE RMSE. A possible reason is that these sequences contain less dynamic interference, so the influence of the proposed optimization is more limited. Even so, the results still indicate that the method can provide stable improvement in several representative cases.

Overall, the results in Table 6-1 demonstrate that the proposed method improves trajectory estimation accuracy on the selected KITTI tracking sequences. The improvement is more evident in sequences with stronger dynamic interference, while in less dynamic scenes the gain is relatively moderate.

Section 6.2.2 Case Analysis and Study

To further examine the effect of the proposed method in a representative dynamic scenario, Sequence 0020 is selected for case analysis. This sequence was also used as an important reference during parameter tuning. Compared with many other sequences, Sequence 0020 contains stronger dynamic interference, especially a large number of surrounding vehicles with relatively high speed. Therefore, it provides a suitable test case for evaluating the

proposed trajectory optimization framework in a challenging traffic environment.

The quantitative results on Sequence 0020 show a clear improvement after optimization. As listed in Table 6-1, the ATE RMSE decreases from 11.7323 m to 6.2936 m after optimization, while the translation error is reduced from 3.2245% to 2.2896%. The RPE RMSE also shows a slight reduction, from 0.2250 m to 0.2246 m. These results indicate that the optimized method can reduce the overall trajectory deviation in this highly dynamic sequence.

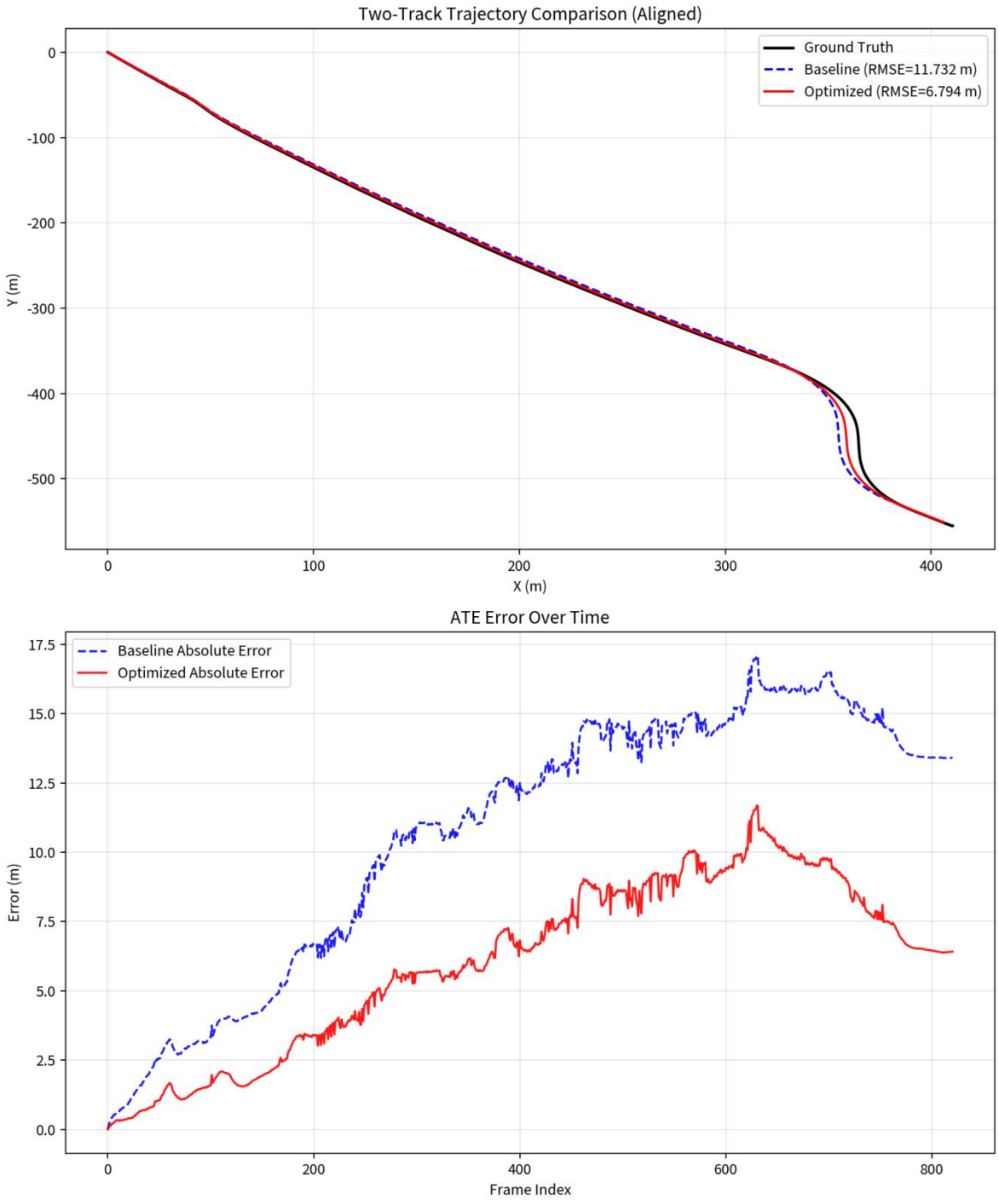

Figure 6-2 Trajectory comparison and ATE error on Sequence 0020

The trajectory comparison in Figure 6-2 further illustrates this improvement. In the aligned trajectory plot, the optimized trajectory remains closer to the ground-truth curve than the baseline, especially in the later part of the sequence

where the road geometry changes more obviously. The deviation of the baseline trajectory becomes more apparent near the turning region, while the optimized result follows the ground truth more closely. This suggests that the proposed method is able to better suppress trajectory drift under dynamic interference.

The lower plot in Figure 6-2 presents the ATE error over time. It can be observed that the baseline error increases continuously as the sequence proceeds, and reaches a relatively high level in the later frames. By contrast, the optimized result maintains a consistently lower error throughout most of the sequence. Although the optimized trajectory still shows some fluctuation, its error accumulation is clearly slower than that of the baseline. This trend is consistent with the reduction in ATE RMSE reported in Table 6-1.

Figure 6-3 Visualization of tracked dynamic vehicles on Sequence 0020

Figure 6-3 provides additional visualization of the dynamic scene in Sequence 0020. As shown in the point cloud and tracking results, multiple

vehicles are distributed around the ego vehicle, and several of them appear in nearby lanes with clear relative motion. Such a scene introduces strong dynamic disturbance to LiDAR-based trajectory estimation. In this case, the proposed framework benefits from the use of tracked dynamic-object information, which helps reduce the negative influence of moving vehicles on trajectory estimation.

Overall, the case study on Sequence 0020 demonstrates that the proposed method is particularly effective in dynamic highway-like traffic scenes with many fast-moving vehicles. Both the quantitative metrics and the visualization results show that the optimized method can produce a trajectory closer to the ground truth and reduce error accumulation over time. This case further supports the effectiveness of the proposed framework for dynamic-scene trajectory estimation.

Chapter 7 Summary and Future Plans

This thesis studies trajectory optimization for FAST-LIO2 in dynamic scenes. Focusing on the influence of moving vehicles on LiDAR-inertial odometry, this work builds a dynamic-scene trajectory optimization framework by integrating 3D object detection, multi-object tracking, front-end dynamic weighting, and back-end trajectory refinement. Within this framework, PointPillars is used to provide 3D object detections, MCTrack is used to maintain temporal consistency of surrounding vehicles, and the tracked-object information is further incorporated into the FAST-LIO2 pipeline for dynamic-scene-aware trajectory estimation.

On the front end, tracked objects are used to identify dynamic regions and assign adaptive weights to LiDAR points, so that the negative influence of moving objects on odometry estimation can be reduced without directly removing dynamic observations. On the back end, a joint optimization module is introduced to refine the ego trajectory by combining odometry consistency and object observation consistency within a sliding window. In addition, a semi-synchronous offline frame scheduling mechanism is designed to improve the stability of the full experimental pipeline. Through these designs, the proposed method extends the original FAST-LIO2 framework to a more structured dynamic-scene trajectory optimization system.

Experiments on KITTI tracking sequences show that the proposed method

can improve trajectory estimation accuracy on several representative sequences. In particular, the improvement is more evident in sequences with stronger dynamic interference, as demonstrated by the quantitative comparison and the case study on Sequence 0020. These results indicate that the proposed framework is effective for reducing trajectory deviation in dynamic driving environments.

Although the current framework achieves encouraging results on some representative sequences, there is still considerable room for further improvement. First, the current PointPillars-based detection module mainly focuses on the forward field of view, which limits the completeness of dynamic-object perception in the full surrounding scene. In future work, the detection range and training strategy can be further improved by modifying the detection setting and retraining the detection network, so that more complete target information can be provided for downstream tracking and trajectory optimization.

Second, the current joint back-end module still has limitations. At present, it mainly refines the ego trajectory, while the tracking results of MCTrack themselves are not further optimized in the back-end process. In addition, the optimized trajectory is currently used only for trajectory evaluation and is not fed back to FAST-LIO2 for map update and front-end refinement. In future work, a tighter coupling strategy can be explored, so that the back-end

optimization result can be further integrated with both target tracking and mapping.

Third, the current experimental evaluation and parameter tuning remain limited. Since the performance of the framework still varies across different KITTI sequences, more systematic experiments are needed in future work to analyze the sensitivity of the method to scene dynamics, tracking quality, and parameter settings. A more comprehensive tuning and validation process would help improve the robustness and generalization ability of the proposed framework.

Overall, this work provides an initial exploration of dynamic-scene trajectory optimization for FAST-LIO2. It demonstrates that tracked dynamic-object information can be incorporated into both front-end and back-end processing to improve trajectory estimation in some challenging driving scenarios. Future improvements in detection, back-end coupling, and experimental validation are expected to further enhance the practicality and stability of the framework.

References

[1] 张顺,黄科薪,甘礼宜,等.基于自动驾驶 SLAM 技术的建图方法研究[J].专用汽车,2025,(02):41-43.DOI:10.19999/j.cnki.1004-0226.2025.02.011.

[2] ZHANG J, SINGH S. LOAM: Lidar Odometry and Mapping in Real-time [C]//Robotics: Science and Systems. 2014.

[3] Xu W, Cai Y, He D, et al. FAST-LIO2: Fast direct LiDAR-inertial odometry[J]. IEEE Transactions on Robotics, 2022, 38(4): 2053-2073. DOI: 10.1109/TRO.2022.3141876.

[4] SMITH R C, CHEESEMAN P C. On the Representation and Estimation of Spatial Uncertainty[J]. The International Journal of Robotics Research, 1986, 5: 56- 68.

[5] 何磊. 面向动态场景的车辆建图与定位方法研究[D]. 上海：上海交通大学，2021.

[6] 危双丰,庞帆,刘振彬等.基于激光雷达的同时定位与地图构建方法综述[J].计算机应用研究，2020，37(02):327-332.

[7] KIM G, KIM A. Remove, then revert: Static point cloud map construction using multiresolution range images[C]// Proceedings of the 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems. Piscataway, NJ: IEEE, 2020: 10758-10765.

[8] LIM H, HWANG S, MYUNG H. ERASOR: Egocentric ratio of pseudo

occupancy-based dynamic object removal for static 3D point cloud map building[J]. IEEE Robotics and Automation Letters, 2021, 6(2): 2272-2279. DOI: 10.1109/LRA.2021.3061363.

[9] Hu X, Yan L, Xie H, Dai J, Zhao Y, Su S. A Novel Lidar Inertial Odometry with Moving Object Detection for Dynamic Scenes[C]//2022 IEEE International Conference on Unmanned Systems. 2022. DOI: 10.1109/ICUS55513.2022.9986661.

[10] Tian X, Zhu Z, Zhao J, Tian G, Ye C. DL-SLOT: Dynamic Lidar SLAM and Object Tracking Based on Collaborative Graph Optimization[EB/OL]. arXiv:2210.15612, 2022.

[11] Peng H, Zhao Z, Wang L. A Review of Dynamic Object Filtering in SLAM Based on 3D LiDAR[J]. Sensors, 2024, 24(2): 645.

[12] MAKHUBELA J K, ZUVA T, AGUNBIADE O Y. A review on vision simultaneous localization and mapping (VSLAM)[C]// 2018 International Conference on Intelligent and Innovative Computing Applications. Piscataway, NJ: IEEE, 2018: 1-5.

[13] HU X, YAN L, XIE H, DAI J, ZHAO Y, SU S. A novel lidar inertial odometry with moving object detection for dynamic scenes[C]// 2022 IEEE International Conference on Unmanned Systems. Piscataway, NJ: IEEE, 2022: 356-361.

[14] Lang A H, Vora S, Caesar H, Zhou L, Yang J, Beijbom O. PointPillars:

Fast Encoders for Object Detection From Point Clouds[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2019: 12697-12705.

[15] Wang X, Qi S, Zhao J, Zhou H, Zhang S, Wang G, Tu K, Guo S, Zhao J, Li J, Yang M. MCTrack: A Unified 3D Multi-Object Tracking Framework for Autonomous Driving[EB/OL]. arXiv:2409.16149, 2024.

[16] WENG X, WANG J, HELD D, KITANI K. 3D multi-object tracking: A baseline and new evaluation metrics[C]// 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems. Piscataway, NJ: IEEE, 2020: 10359-10366.

[17] GEIGER A, LENZ P, STILLER C, et al. Vision meets robotics: The KITTI dataset[J]. International Journal of Robotics Research, 2013.